

Bayesian Clustering with User-Side Information for Streaming Recommender Systems: Addressing Data Sparsity and Cold-Start with Provable Convergence

July 3, 2026

Abstract

Collaborative filtering, a popular technique in recommendation systems, faces primary challenges: data sparsity, cold-start limitations, and an inability to adapt to streaming data in real-world environments. Although existing heuristic and distance-based approaches partially address streaming scenarios, they lack theoretical rigor and fail to leverage evolving user-item interactions effectively. This study introduces a novel Bayesian clustering framework that integrates user-side information with probabilistic priors to overcome these challenges. The proposed clustering method reduces the impact of rating matrix sparsity by leveraging prior knowledge for streaming data. Furthermore, incorporating user-side information mitigate the effects of the cold-start problem on prediction accuracy. The convergence of the proposed clustering method is established in this study through the proof that the Hessian matrix of its objective function is positive definite. Empirical validation demonstrates a faster convergence rate of proposed method compared to the Fuzzy C-Means. The experimental results on real-world streaming datasets (MovieLens, yelp) indicate that the proposed method, in addition to overcoming the limitations, achieves higher accuracy in predicting user ratings than other methods. By integrating Bayesian inference with the ability to adapt to streaming data, this study addresses key limitations in current recommender systems research.

Keywords: Bayesian Clustering, Streaming Data, Collaborative Filtering, Cold-Start Mitigation, Data Sparsity, Convergence Analysis, Side Information, Recommendation Systems.

1 Introduction

Every piece of knowledge builds upon another, and the understanding of any truth requires awareness of its foundation. Recommender systems, much like the evolution of knowledge, build upon prior information to refine and enhance decision-making. These systems aim to maximize user satisfaction by generating a list of the most relevant and personalized items. Given their significance, extensive research has been conducted in this domain, exploring various methodologies to improve recommendation quality. Each approach relies on different sources of information, benefiting from past interactions to predict future preferences. However, much like gaps in historical knowledge challenge understanding, recommender systems also struggle with incomplete data, cold-start issues, and the dynamic nature of evolving user-item interactions.

Recommender systems aim to enhance user satisfaction by generating a list of the most relevant and personalized items. Given their significance, extensive research has been conducted in this domain, exploring various methodologies for improving recommendation quality. Each approach leverages different types of information and comes with its own advantages and limitations.

challenge 1: Despite substantial advancements, recommender systems still face critical challenges rooted in the nature of user-item interaction data, including the cold-start problem, data sparsity, and imbalance **li2024recent, rama2019deep**. These limitations reduce prediction accuracy, preventing the system from effectively tailoring recommendations to user preferences.

challenge 2: Many existing models assume a static dataset, whereas in real-world scenarios, data arrives continuously in a streaming manner. The ability to handle such dynamic data influx remains a pressing challenge, as most traditional approaches lack adaptability to evolving user-item interactions.

To address these limitations, this paper proposes a Bayesian clustering framework for streaming recommendation that addresses sparsity and cold-start by leveraging user-side information. we derive an update rule with provable convergence guarantees, ensuring that the model adapts stably as the data stream evolves. This model enhances both the accuracy and adaptability of recommender systems.

The remainder of this paper is organized as follows. Section 2 reviews the literature on content-based, collaborative methods, with a detailed focus on clustering-based CF and the use of side information. Section 3 details the proposed method, including the clustering techniques employed and their integration into the recommender system. Section 4 presents the evaluation of the system’s performance, offering a comparative analysis with existing methods. Finally, Section 5 concludes the study and suggesting directions for future research.

2 Related Work

Methods in recommender systems are included three main categories content-based methods, collaborative Filtering(CF), and Hybrid methods **aljunid2025collaborative**. Content-based methods generate recommendations solely based on a user’s past behavior and interests, without considering other users’ interactions. While simple to implement, they provide a limited selection of similar items and fail to leverage relationships between users.

Collaborative filtering focuses on user-item interactions rather than item characteristics, recommending items based on the preferences of similar users. This approach enables more diverse and personalized recommendations by utilizing shared user behaviors and preferences **isinkaye2023matrix**. Collaborative methods have two types of algorithms, memory-based and model-based.

In memory-based methods, which are also called neighborhood-based methods, the relationship between users and items is investigated. User-based **zhao2010user**, **valcarce2019collaborative** or item-based **sarwar2001item**, **fu2018attention** similarity is considered to predict the user’s rating of items that he has not seen before. The primary limitation of memory-based methods is their reliance on loading the entire rating matrix into memory for each prediction, making them inefficient for large-scale datasets in terms of time and memory usage.

Model-based methods address this issue by employing a predictive model based on past user-item interactions. Various machine learning algorithms can be utilized in model-based approaches to enhance recommendation accuracy and efficiency. These mainly includes approaches like Matrix factorization **isinkaye2023matrix**, **shi2020user**, **yang2018enhancing**, **behera2023mixed**, Bayes classifier **valdiviezo2019collaborative**, neural networks **wang2015collaborative**, **angel2022deep**, **li2017collaborative**. This paper focuses on common challenges in recommender systems and clustering algorithms. Clustering techniques are commonly used to mitigate the sparsity problem and improve the scalability of CF methods. By grouping users or items with similar characteristics, clustering-based CF systems enhance recommendation performance while reducing computational complexity.

a) Clustering-based Collaborative Filtering

Clustering algorithms such as k-means **dakhel2011new**, **zahra2015novel**, **ranjan2018category**, **kant2017leaderrank** and FCM **birtolo2013advances** are widely utilized in recommender systems. Studies have explored various user clustering approaches, including Bayesian matrix factorization **bobadilla2018recommender**, evolutionary clustering that adapts user states over time **liji2018improved**, and weighted clustering that assigns higher importance to users with more interactions **salah2015dynamic**. In **katarya2018recommender**, a hybrid method used gray wolf optimization for initializing clusters, followed by fuzzy c-means (FCM) for user classification based on rating similarity . Similarly, for item clustering, in **deng2019novel**, an improved Kullback-Leibler divergence is employed to measure item similarity based on probability distributions. In **kant2017leaderrank**, the k-means algorithm is utilized to cluster items, and an item’s score is predicted solely based on the average scores of other items within the same cluster. Several methods have been proposed for clustering both users and items. Other approaches include power fuzzy clustering using an exponential fitness function to refine membership **treerattanapitak2012exponential** and network-based clustering where evolving statuses lead to stable groups **chen2017evolutionary**. Clustering via the QUBIC (A Qualitative Biclustering algorithm) in **singh2018impact**, aim to group similar users and items effectively. **science2018dynamic**

constructs a network that integrates user characteristics, item features, and temporal information, subsequently applying a dynamic evolutionary clustering algorithm.

Several advanced clustering-aware approaches such as **rajput2025nsga**, proposes an NSGA-II-optimized deep autoencoder framework improve multi-criteria recommendations by optimizing model parameters through a multi-objective approach, addressing data sparsity and non-linearity. **li2025self** presents a self-supervised graph Transformer (SGTN) that improves user-item representations by encoding ratings and applying contrastive learning. It focuses on capturing social and collaborative relationships through decentralized graphs and edge-aware attention. Although not explicitly designed for clustering, it enhances user embeddings, enabling dynamic and context-sensitive user-item grouping in recommendation systems. In **qian2025ifm** a framework that uses adversarial features from multiple models is proposed to improve transferability and enhance user-item representations. It refines embeddings by reducing noise and improving cluster separability, offering a strategy for robust clustering in dynamic recommendation systems. **he2025fedai** presents FedAI, a federated recommendation system that combines spectral clustering and K-means to group users while ensuring privacy. High-order interactions enhance embeddings, improving clustering in decentralized settings without accessing raw data.

b) Clustering based on User-side Information

While clustering-based methods address scalability and sparsity issues in the score matrix, the high dimensionality of user-item data makes them computationally expensive. To mitigate this, many approaches leverage side information as clustering features, significantly reducing the feature space and improving efficiency. **liji2018improved** employs user attributes such as age, gender, and residence to calculate feature distances for clustering. Similarly, **chen2019n2vscdnr** forms a bipartite user-item graph and converts each node into a vector before clustering. Additionally, **zhao2017coarse** clusters users based on social network labels and items via a bipartite graph, **sheugh2018novel** creates a two-dimensional similarity graph of users for clustering, and **wasid2018clustering** incorporates social information alongside the score matrix to enhance user clustering. Gong et al. **gong2010collaborative** leveraged both explicit ratings and implicit user behavior to improve similarity measurements. Lu et al. **lu2021novel** addressed the sparsity problem in user-item interactions by grouping items with similar features, thereby aligning recommendations more closely with user interests. Additionally, Wasid et al. **wasid2018improved** refined traditional collaborative filtering through a multi-criteria clustering approach, which integrates various aspects of user preferences to yield more diverse and accurate recommendations. Additionally, Mahalanobis distance is employed to measure the similarity between users' score vectors, taking into account not only the given scores but also their variance and covariance **deng2019neural** used variational autoencoders and side information to mitigate data sparsity and the cold start problem.

c) Recommendation in Streaming Data Environment

Furthermore, most of clustering-based methods often do not consider streaming data, making them less effective in dynamic environments. With the increasing need for real-time recommendations, online recommender systems have become essential for handling streaming data. These systems must efficiently update user preferences and item interactions as new information becomes available. Recommender systems in e-commerce and digital service platforms must continuously process new users and items, making real-time data handling essential. However, most existing systems rely on fixed datasets for evaluation, limiting their applicability in dynamic environments. Traditional content-based and memory-based collaborative filtering methods require loading all data into memory, making them unsuitable for streaming data. In contrast, model-based collaborative filtering, particularly matrix decomposition methods like ALS and SGD, allows incremental updates. While these methods address some streaming challenges, they remain computationally expensive and risk losing information during model updates. Clustering-based recommender systems for streaming data mainly rely on heuristics without leveraging prior knowledge, limiting their effectiveness. For instance, distance-based clustering adjusts noisy data positioning **georgieva2008dynamic**, as well as a density-based method updates clusters dynamically using local averages **baruah2012evolving**. In **das2007google** an online model proposed for Google News, continuously updates recommendations based on user interactions,

ensuring that personalized news suggestions remain relevant. A streaming matrix-factorization framework proposed in **jakomin2020simultaneous** that incrementally fuses multiple heterogeneous data streams to continuously adapt latent user-item representations and mitigate both data sparsity and cold-start issues. While online recommender systems rely on user feedback, new users, and new items for updates, many clustering-based methods struggle with streaming data. Integrating online learning into clustering-based collaborative filtering can improve adaptability in dynamic environments.

One of the primary strategies for mitigating data sparsity is cluster-based collaborative filtering. Clustering-based recommender systems can enhance recommendation reliability while maintaining lower computational complexity. Despite the benefits of clustering-based CF methods, challenges remain. Feature selection for clustering is a critical issue, as high-dimensional user-item matrices can lead to the curse of dimensionality. Conversely, clustering based on side features alone may not accurately capture user interests, reducing prediction accuracy. In addition, deal with streaming data, making them effective in dynamic environments is rally important.

To address these limitations, a Bayesian clustering model is introduced, incorporating prior knowledge for streaming data clustering. Unlike traditional clustering approaches, this method dynamically updates user and item clusters as new interactions occur. By combining user ratings with side features, the model improves personalization, incorporates real-time updates to handle user feedback, new users, and alleviates the cold start problem by leveraging additional user information beyond the rating matrix. Experimental results demonstrate that this approach outperforms existing clustering-based CF models in terms of accuracy and efficiency, making it well-suited for real-time recommendation systems.

3 Proposed Method

3.1 Problem Formulation through Bayesian Latent Filtering

Let $x \in \mathcal{X}$ denote an incoming item. The goal is to infer which candidate item-set

$$\mathcal{S}_j \subseteq \mathcal{X}, \quad j = 1, \dots, K,$$

is most relevant to x . Each set $\mathcal{S}_j$ is represented by a latent descriptor

$$\theta_j = (\mu_j, \Sigma_j),$$

where μ_j and Σ_j describe the center and dispersion of the latent representation of the items belonging to $\mathcal{S}_j$.

Instead of directly mapping x to $\mathcal{S}_j$, we introduce an intermediate latent filtering layer. First, the incoming item is encoded into a latent vector

$$\mathbf{z}(x) = \phi(x) \in \mathbb{R}^d.$$

Then, the model estimates the posterior involvement of x in each hidden component c_i , $i = 1, \dots, C$:

$$q_i(x) \triangleq P(c_i | x) = P(c_i | \mathbf{z}(x)).$$

This posterior plays the role of a Bayesian filter over the latent space.

To compute $q_i(x)$, each hidden component c_i is represented by a latent prototype and covariance

$$\psi_i = (\mathbf{m}_i, \Lambda_i),$$

and the likelihood of the encoded item under component c_i is modeled as

$$p(\mathbf{z}(x) | c_i) = \mathcal{N}(\mathbf{z}(x); \mathbf{m}_i, \Lambda_i).$$

Hence, by Bayes' rule,

$$q_i(x) = P(c_i | x) = \frac{\rho_i \mathcal{N}(\mathbf{z}(x); \mathbf{m}_i, \Lambda_i)}{\sum_{\ell=1}^C \rho_\ell \mathcal{N}(\mathbf{z}(x); \mathbf{m}_\ell, \Lambda_\ell)},$$

where $\rho_i = P(c_i)$ is the prior probability of the i -th hidden component.

After this filtering step, the relevance of the hidden component c_i to the candidate set $\mathcal{S}_j$ is evaluated through the latent descriptor θ_j . In particular, the prototype $\mathbf{m}_i$ is assessed under the distribution associated with $\mathcal{S}_j$:

$$f(\mathbf{m}_i | \theta_j) = \mathcal{N}(\mathbf{m}_i; \mu_j, \Sigma_j).$$

The correct Gaussian form is

$$f(\mathbf{m}_i | \theta_j) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{m}_i - \mu_j)^T \Sigma_j^{-1} (\mathbf{m}_i - \mu_j) \right).$$

Therefore, using Bayes' rule, the posterior association between the set descriptor θ_j and the hidden component c_i is

$$f(\theta_j | \mathbf{m}_i) = \frac{f(\mathbf{m}_i | \theta_j) P(\theta_j)}{\sum_{r=1}^K f(\mathbf{m}_i | \theta_r) P(\theta_r)}.$$

The complete inference chain is therefore

$$x \longrightarrow \mathbf{z}(x) = \phi(x) \longrightarrow P(c_i | x) \longrightarrow f(\theta_j | \mathbf{m}_i) \longrightarrow \mathcal{S}_j.$$

Accordingly, the relevance score of the incoming item x to the candidate set $\mathcal{S}_j$ is obtained by marginalizing over all hidden components:

$$R_j(x) = \sum_{i=1}^C P(c_i | x) f(\theta_j | \mathbf{m}_i).$$

Finally, the most relevant set is estimated as

$$\hat{j} = \arg \max_j R_j(x) = \arg \max_j \sum_{i=1}^C P(c_i | x) f(\theta_j | \mathbf{m}_i),$$

and the recommended set is

$$\hat{\mathcal{S}}(x) = \mathcal{S}_{\hat{j}}.$$

This formulation separates the two probabilistic roles of the latent space. The term $P(c_i | x)$ measures how the incoming item activates hidden components, while $f(\theta_j | \mathbf{m}_i)$ measures how each hidden component supports a candidate recommendation set.

3.2 Bayesian Latent Filtering View

Let $x \in \mathcal{X}$ denote an incoming item. The purpose of the recommender is to estimate the most relevant candidate set

$$\mathcal{S}_j \subseteq \mathcal{X}, \quad j = 1, \dots, K,$$

for the input item x . Each set $\mathcal{S}_j$ is represented by a latent descriptor

$$\theta_j = (\mu_j, \Sigma_j).$$

We introduce a hidden component variable

$$c_i, \quad i = 1, \dots, C,$$

where c_i denotes the i -th internal latent cluster or hidden component. Each hidden component is represented by a prototype

$$\mathbf{m}_i \in \mathbb{R}^d.$$

Thus, c_i is the latent event, while $\mathbf{m}_i$ is its continuous representative in the latent space.

The input item is first mapped into a latent representation

$$\mathbf{z}(x) = \phi(x).$$

The Bayesian filtering stage estimates the posterior activation of each hidden component:

$$q_i(x) = P(c_i | x) = \frac{\rho_i p(\mathbf{z}(x) | c_i)}{\sum_{\ell=1}^C \rho_\ell p(\mathbf{z}(x) | c_\ell)},$$

where $\rho_i = P(c_i)$ is the prior probability of the i -th hidden component.

If the latent representation of component c_i is modeled by a Gaussian distribution centered at $\mathbf{m}_i$, then

$$p(\mathbf{z}(x) | c_i) = \mathcal{N}(\mathbf{z}(x); \mathbf{m}_i, \Lambda_i).$$

After the hidden component is inferred, the model evaluates how strongly this internal component supports each candidate set $\mathcal{S}_j$. This is expressed as

$$f(\theta_j | c_i).$$

Since c_i is represented by its prototype $\mathbf{m}_i$, this posterior can be computed through

$$f(\theta_j | c_i) \equiv f(\theta_j | \mathbf{m}_i).$$

Using Bayes' rule, we obtain

$$f(\theta_j | \mathbf{m}_i) = \frac{f(\mathbf{m}_i | \theta_j) P(\theta_j)}{\sum_{r=1}^K f(\mathbf{m}_i | \theta_r) P(\theta_r)}.$$

Here,

$$f(\mathbf{m}_i | \theta_j) = \mathcal{N}(\mathbf{m}_i; \mu_j, \Sigma_j),$$

that is,

$$f(\mathbf{m}_i | \theta_j) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{m}_i - \mu_j)^T \Sigma_j^{-1} (\mathbf{m}_i - \mu_j) \right).$$

Therefore, the relevance score of the incoming item x to the candidate set $\mathcal{S}_j$ is

$$R_j(x) = \sum_{i=1}^C P(c_i | x) f(\theta_j | c_i).$$

Finally, the estimated recommendation set is

$$\hat{\mathcal{S}}(x) = \mathcal{S}_{\hat{j}}, \quad \hat{j} = \arg \max_j \sum_{i=1}^C P(c_i | x) f(\theta_j | c_i).$$

Let $x \in \mathcal{X}$ denote an incoming item for which the system must identify the most relevant item-set among

$$\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_K,$$

where each $\mathcal{S}_j \subseteq \mathcal{X}$ represents a set of items sharing a latent recommendation relation. In this setting, the objective is not merely to assign x to a precomputed cluster, but to infer the relevance of x to each candidate set $\mathcal{S}_j$ under uncertainty and incomplete observations.

To formalize this relation, we introduce a latent variable

$$z_i, \quad i = 1, \dots, C,$$

which represents an intermediate hidden state between the incoming item x and the candidate recommendation sets. This latent layer plays an encoder-like role: instead of directly mapping x to a recommendation set, the model first estimates the posterior involvement of x in the hidden states and then decodes this latent evidence into set-level relevance scores. The resulting probabilistic chain can be written as

$$x \longrightarrow P(z_i | x) \longrightarrow f(\theta_j | z_i) \longrightarrow \mathcal{S}_j,$$

where θ_j denotes the latent descriptor or representative parameter of the set $\mathcal{S}_j$.

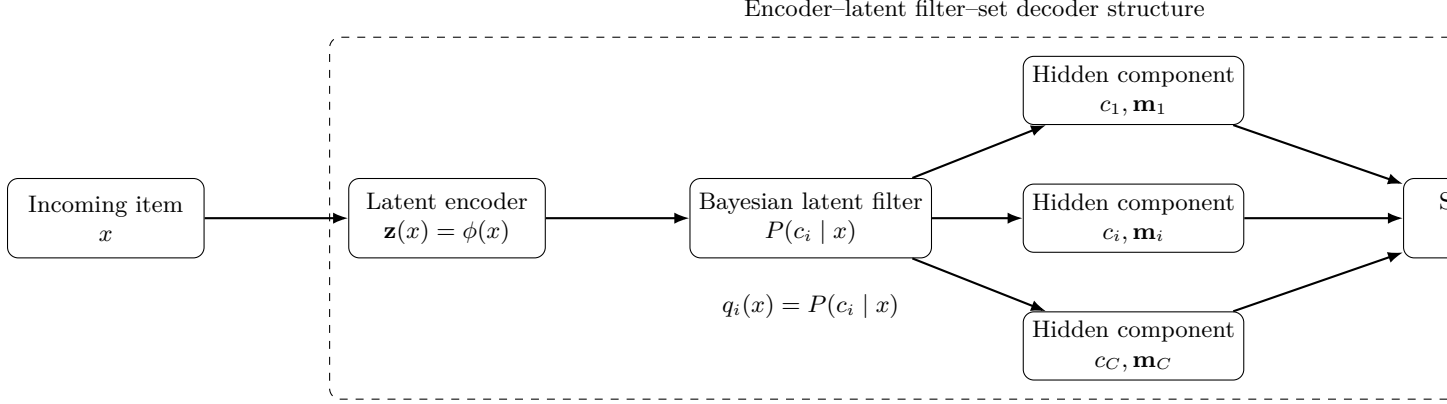


Figure 1: Bayesian latent filtering view of the proposed recommender. The structure is encoder-like because the item x is first mapped to a latent representation and then filtered through hidden components. It is not a classical autoencoder, since the objective is not reconstruction of x , but inference of the most relevant recommendation set $\hat{\mathcal{S}}(x)$.

3.3 From General Relevance Scoring to a Tractable Streaming Instantiation

The formulation of Section ?? assumes the hidden-component parameters $\psi_i = (\mathbf{m}_i, \Lambda_i)$ and the set descriptors $\theta_j = (\mu_j, \Sigma_j)$ are already available, so that $P(c_i | x)$ and $f(\theta_j | c_i)$ can be evaluated directly via Eqs. (??) and (??). In a real recommender, however, neither is given in advance: both the latent prototypes and the candidate-set descriptors must be estimated from streaming user/item data.

We therefore instantiate the general framework in the regime where hidden components and candidate sets coincide, $C = K$ and $c_j \equiv \mathcal{S}_j$, so that the two-stage relevance score of Eq. (??) collapses to a single learned filter,

$$R_j(x) = f(\theta_j | \mathbf{z}(x)), \quad (1)$$

and the estimated-set rule of Eq. (??) is unchanged, $\hat{\mathcal{S}}(x) = \mathcal{S}_{\hat{j}}$ with $\hat{j} = \arg \max_j R_j(x)$. Because θ_j is now unknown, the posterior $P(c_j | x)$ cannot be evaluated in closed form as in Eq. (??); instead, it is *jointly estimated* together with θ_j through a relaxed, fuzzy surrogate $u_{ij} \in [0, 1]$ that plays the same role as $P(c_j | x_i)$ but is optimized rather than computed. This yields the maximum a posteriori training objective

$$\max_{u, \theta} \sum_{j=1}^K \sum_{i=1}^N u_{ij}^m R_j(x_i), \quad 1 < m < \infty,$$

which is exactly the quantity developed below, first for fixed items (Section 3.4) and then for streaming users, where the candidate-set descriptors θ_j are updated online as new evidence arrives (Section 3.5). Sections 3.6–3.8 extend this instantiation with graph-based smoothing of the estimated θ_j , a rating-prediction rule built directly on $R_j(x)$, and convergence guarantees for the resulting alternating estimator.

3.4 Prototype Discovery via Fuzzy Item Clustering

As a first, static instantiation of Eq. (1), items are assumed fixed and are clustered using the Fuzzy C-Means (FCM) algorithm, which recovers a set of item-side prototypes analogous to $\{\mathbf{m}_i\}$ in the general framework. The objective function is

$$J = \sum_{k=1}^{C_i} \sum_{i=1}^n u_{ki}^m \|x_i - \mu_k\|^2, \quad (2)$$

where $u_{ki} \in [0, 1]$ is the degree of membership of item x_i in cluster k , μ_k is the corresponding centroid, n is the number of items, C_i is the number of item clusters, and $m > 1$ is the fuzzifier. Differentiating J with respect to u and μ yields the standard closed-form membership and centroid updates. These item-side clusters serve as fixed reference prototypes for the streaming user-side instantiation developed next.

3.5 Streaming Bayesian Realization of the Set Descriptors

Users, unlike items, arrive continuously, so the set descriptors θ_j of Eq. (1) cannot be estimated once and fixed; they must be realized online. Rather than retaining all historical data, the proposed method maintains, for each candidate set j , only its current descriptor $\theta_j = (\mu_j, \Sigma_j)$, updated incrementally as new users are observed — an online, memory-efficient realization of $R_j(x)$.

3.5.1 Objective Function

Let $z_i = \phi(x_i) \in \mathbb{R}^d$ denote the latent feature vector of user i (Section ??), formed by concatenating their rating vector and side-feature vector,

$$\mu_j = \langle [r_{\mu j 1}, r_{\mu j 2}, \dots, r_{\mu j C}; f_{\mu j 1}, \dots, f_{\mu j K}] \rangle.$$

Substituting the collapsed relevance score $R_j(x_i) = f(\theta_j | z_i)$ into the training objective of Section 3.3 gives

$$\max_{u, \theta_j} \sum_{j=1}^C \sum_{i=1}^N u_{ij}^m f(\theta_j | z_i), \quad 1 < m < \infty. \quad (3)$$

By Bayes' theorem, Eq. (3) is rewritten as

$$\max_{u, \theta_j} \sum_{j=1}^C \sum_{i=1}^N u_{ij}^m f(z_i | \theta_j) f(\theta_j), \quad 1 < m < \infty, \quad (4)$$

where $f(z_i | \theta_j)$ is the likelihood of user i 's latent vector under set j 's descriptor, and $f(\theta_j)$ is the prior over that descriptor — the same two Bayesian ingredients as $p(\mathbf{z}(x) | c_i)$ and ρ_i in Section ??, now instantiated for a descriptor that must itself be estimated. Assuming both are Gaussian,

$$f(z_i | \theta_j) \sim \mathcal{N}(z_i; \mu_j, \Sigma_j) = \frac{1}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp\left(-\frac{1}{2}(z_i - \mu_j)^T \Sigma_j^{-1} (z_i - \mu_j)\right), \quad (5)$$

$$f(\theta_j) \sim \mathcal{N}(\mu_j; \mu_{\theta_j}, \Sigma_{\theta_j}) = \frac{1}{(2\pi)^{d/2} |\Sigma_{\theta_j}|^{1/2}} \exp\left(-\frac{1}{2}(\mu_j - \mu_{\theta_j})^T \Sigma_{\theta_j}^{-1} (\mu_j - \mu_{\theta_j})\right), \quad (6)$$

where μ_{θ_j} and Σ_{θ_j} are the mean and covariance of the prior over θ_j , carried over from the previous streaming step. Substituting Eqs. (5)–(6) into (4) gives the streaming objective

$$\begin{aligned} J_m(u_{11}, \dots, u_{NC}, \theta_1, \dots, \theta_C) &= \sum_{j=1}^C \sum_{i=1}^N \frac{u_{ij}^m}{2\pi |\Sigma_j|^{1/2} |\Sigma_{\theta_j}|^{1/2}} \\ &\times \exp\left(-\frac{1}{2}\left((z_i - \mu_j)^T \Sigma_j^{-1} (z_i - \mu_j) + (\mu_j - \mu_{\theta_j})^T \Sigma_{\theta_j}^{-1} (\mu_j - \mu_{\theta_j})\right)\right). \end{aligned} \quad (7)$$

3.5.2 Update Equations

Incorporating the constraint $-\sum_{i=1}^N \lambda_i (\sum_{j=1}^C u_{ij} - 1)$ via a Lagrangian and differentiating with respect to each parameter yields the update rules below. For clarity, define the auxiliary quantities

$$f_{ij} = \exp\left(-\frac{1}{2}\left((z_i - \mu_j)^T \Sigma_j^{-1} (z_i - \mu_j) + (\mu_j - \mu_{\theta_j})^T \Sigma_{\theta_j}^{-1} (\mu_j - \mu_{\theta_j})\right)\right), \quad (8)$$

$$S = \frac{\Sigma_j^{-1}}{\Sigma_j^{-1} + \Sigma_{\theta_j}^{-1}}, \quad 1 - S = \frac{\Sigma_{\theta_j}^{-1}}{\Sigma_j^{-1} + \Sigma_{\theta_j}^{-1}}. \quad (9)$$

The set descriptor's mean is updated as a Bayesian-weighted combination of the evidence-driven estimate and the prior,

$$\mu_j = S \cdot \frac{\sum_{i=1}^N u_{ij}^m f_{ij} z_i}{\sum_{i=1}^N u_{ij}^m f_{ij}} + (1 - S) \cdot \mu_{\theta_j}, \quad (10)$$

the covariance as

$$\Sigma_j = \frac{\sum_{i=1}^N u_{ij}^m (z_i - \mu_j)(z_i - \mu_j)^T f_{ij}}{\sum_{i=1}^N u_{ij}^m f_{ij}}, \quad (11)$$

and the fuzzy surrogate for $P(c_j | x_i)$ — the learned analogue of Eq. (??) — as

$$u_{ij} = \left(\sum_{k=1}^C \left(\frac{f_{ij}}{f_{kj}} \right)^{\frac{1}{m-1}} \right)^{-1}, \quad u_{ij} \in [0, 1]. \quad (12)$$

The prior for the next streaming step is then carried forward as

$$\mu_{\theta_j}^{k+1} = \mu_j^k, \quad (\Sigma_{\theta_j}^{k+1})^{-1} = \frac{\sum_{i=1}^N u_{ij}^m (\mu_j - \mu_{\theta_j})(\mu_j - \mu_{\theta_j})^T f_{ij}}{\sum_{i=1}^N u_{ij}^m f_{ij}}, \quad (13)$$

where superscripts $k, k+1$ index consecutive streaming iterations.

In the resulting iterative procedure, a random membership matrix is initialized; set descriptors are computed from it via Eqs. (10)–(11); the membership matrix is updated accordingly via Eq. (12); and this alternation continues until the change in membership values between consecutive iterations falls below a tolerance ϵ . When a new user $N+1$ arrives, the descriptors are updated by incorporating that single user via Eqs. (10)–(11), and its membership degrees follow from Eq. (12) — without reprocessing any previously seen user. Algorithm 1 summarizes this realization of the general latent-filtering framework.

Algorithm 1 Streaming Realization of the Latent Filter via Bayesian User Clustering

- 1: Initialize a random membership matrix $u^{(0)} \in \mathbb{R}^{N \times C}$
 - 2: **repeat**
 - 3: **for** each candidate set $j = 1, \dots, C$ **do**
 - 4: Update descriptor $\theta_j = (\mu_j, \Sigma_j)$ and prior $(\mu_{\theta_j}^k, \Sigma_{\theta_j}^k)$ via Eqs. (10), (11), (13)
 - 5: **end for**
 - 6: Update membership matrix $u^{(k+1)}$ via Eq. (12)
 - 7: **until** $\max_{i,j} |u_{ij}^{(k)} - u_{ij}^{(k+1)}| < \epsilon$
 - 8: **Output:** set descriptors (μ_j, Σ_j) and user membership degrees u_{ij} , jointly realizing $\hat{\mathcal{S}}(x) = \mathcal{S}_{\hat{j}}, \hat{j} = \arg \max_j u_{ij}$
-

3.6 Graph-Regularized Prototype Propagation

Recommender systems typically have access to user features, selection history, user–item ratings, and item attributes, but users are also influenced by socially or behaviorally similar peers. To exploit this, the streaming realization above is extended with a neighborhood graph over set descriptors $\{\mu_j\}$: descriptors whose pairwise distance falls below a threshold are connected as neighbors (Fig. 2). A *community* is a subgraph with higher internal than external edge density; descriptors within the same community are permitted to exert more mutual influence than descriptors in different communities.

After each standard update (Eq. (10)), every centroid is additionally smoothed toward its graph neighbors, proportionally to distance:

$$\hat{\mu}_j = \frac{\sum_{l=1}^{N_j} w_l \mu_l}{\sum_{l=1}^{N_j} w_l}, \quad w_l = \exp(-\beta d_{jl}), \quad (14)$$

where $\hat{\mu}_j$ is the smoothed descriptor mean for set j , N_j is its number of graph neighbors, μ_l is neighbor l ’s mean, d_{jl} is the feature-space distance between descriptors j and l , and $\beta > 0$ is a dataset-specific damping constant preventing any single neighbor from dominating the update. This regularization limits the impact of outlier evidence on any one descriptor, controlling abrupt changes and yielding a more robust streaming realization of θ_j .

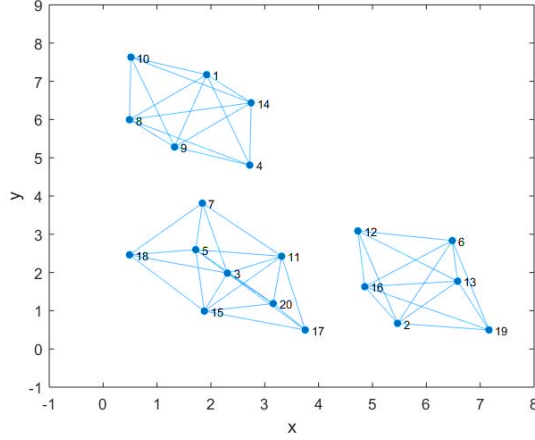


Figure 2: An example of the neighborhood graph representation among set-descriptor centroids.

3.7 Latent-Filtered Rating Prediction and Top- N Recommendation

Given a target user’s membership degrees u_{ij} — either previously computed (existing user) or freshly computed via Eq. (12) (new user) — the predicted rating for a candidate item is obtained by averaging the ratings assigned to that item by the user’s neighbors within the sets to which they belong with non-negligible membership, directly instantiating the relevance-driven selection rule of Eq. (??):

$$\hat{r} = \frac{1}{C \times M} \sum_{j=1}^C \sum_{l=1}^M r_{jl}, \quad (15)$$

where C is the number of sets the target user belongs to with non-negligible membership and M is the number of neighbors considered per set. After predicting scores for all candidate items, the top- N items by predicted rating are recommended; following common practice, we use $N = 5$.

3.8 Theoretical Analysis

3.8.1 Convergence of the Streaming Estimator

The objective J_m (Eq. (7)) is optimized by alternating between updating the membership matrix u (Eq. (12)) and updating the set descriptors θ_j (Eqs. (10)–(11)), holding the other block fixed at each step — a block-coordinate (alternating) minimization scheme in the same family as classical FCM. Convergence follows from the generalized convergence theorem of Zangwill together with the local-convergence result of **hathaway1986local**: provided (i) each subproblem — minimizing J_m over u with θ fixed, and over θ with u fixed — has a unique, continuous solution (guaranteed here, since Eq. (12) and Eqs. (10)–(11) are closed-form and continuous in their inputs), and (ii) the iterates are initialized sufficiently close to a stationary point of J_m , the sequence $\{u^{(k)}, \theta^{(k)}\}$ converges to a local minimum of J_m .

Condition (ii) motivates the initial (non-streaming) clustering pass in Section 3.4: performing item-side clustering before entering the streaming loop places the iterates close to a minimum of J_m from the outset, rather than relying on convergence from an arbitrary random start.

3.8.2 Convergence Rate

We next characterize how quickly a set descriptor’s mean stabilizes under the streaming update. From Eq. (10), incorporating a new sample z_{n+1} into set j yields the recursion

$$\mu_{n+1} = S \mu_n + (1 - S) z_{n+1} + M, \quad (16)$$

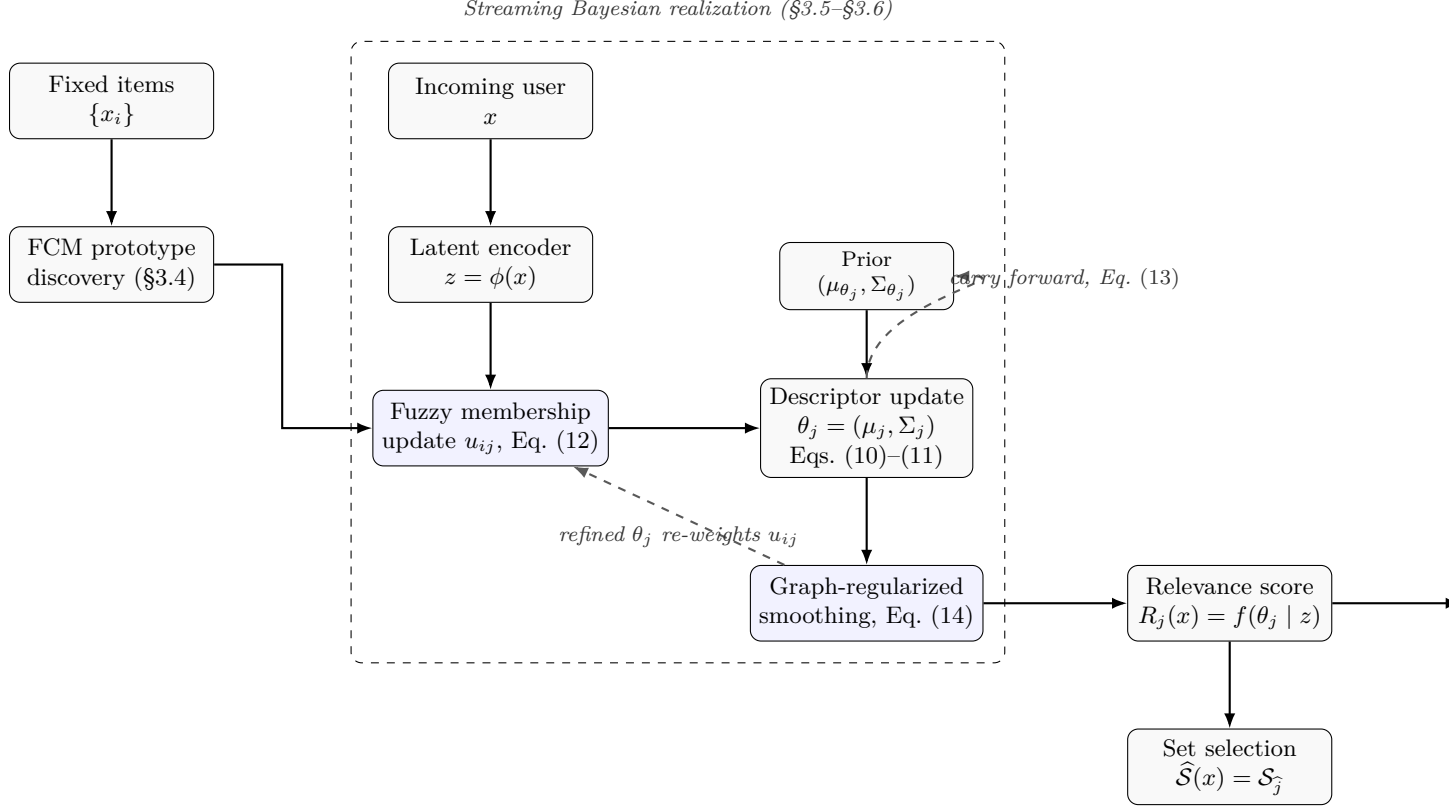


Figure 3: Streaming latent-filtering recommendation pipeline. Item-side prototypes are discovered once, offline (left), while user-side set descriptors θ_j are realized online through an alternating Bayesian loop: incoming users update fuzzy memberships u_{ij} and descriptors θ_j , which are graph-regularized and fed back as priors for the next arrival (dashed feedback arrows). The resulting descriptors instantiate the general relevance score $R_j(x)$ of Eq. (1), used both for set selection and for rating-based top- N recommendation.

where, from Eqs. (10)–(9), $S = \sum_{i=1}^n u_{ij}^m f_{ij} / \sum_{i=1}^{n+1} u_{ij}^m f_{ij} \in [0, 1)$ is the fraction of accumulated evidence mass contributed by previously seen samples, and M collects the bounded prior-pull term.

Equation (16) is a first-order linear recursion, i.e., a discrete-time contraction whenever $S \in [0, 1)$. Unrolling it k steps from an initial mean $\mu(0)$ gives the closed form

$$\mu(k) = S^k \mu(0) + \sum_{i=1}^k S^{k-i} (1 - S) z(i) + \sum_{i=1}^k S^{k-i} M(i). \quad (17)$$

Let μ^* denote the fixed point of the recursion. Subtracting the fixed-point relation from Eq. (16) gives

$$\mu(k) - \mu^* = S^k (\mu(0) - \mu^*) + O(1 - S), \quad (18)$$

so the deviation from the fixed point decays geometrically with rate S : $\|\mu(k) - \mu^*\| = O(S^k)$. Because S is the fraction of evidence mass attributable to *previously accumulated* samples, S is smaller — and hence convergence is faster — precisely when the streaming update weighs incoming evidence more heavily relative to accumulated history, the regime induced by the proposed Bayesian weighting in Eq. (9). This gives a direct, closed-form account of the faster empirical convergence of the proposed method relative to standard FCM (Fig. 5): both follow a recursion of the form of Eq. (16), but differ in the effective contraction rate S induced by their respective weighting schemes.

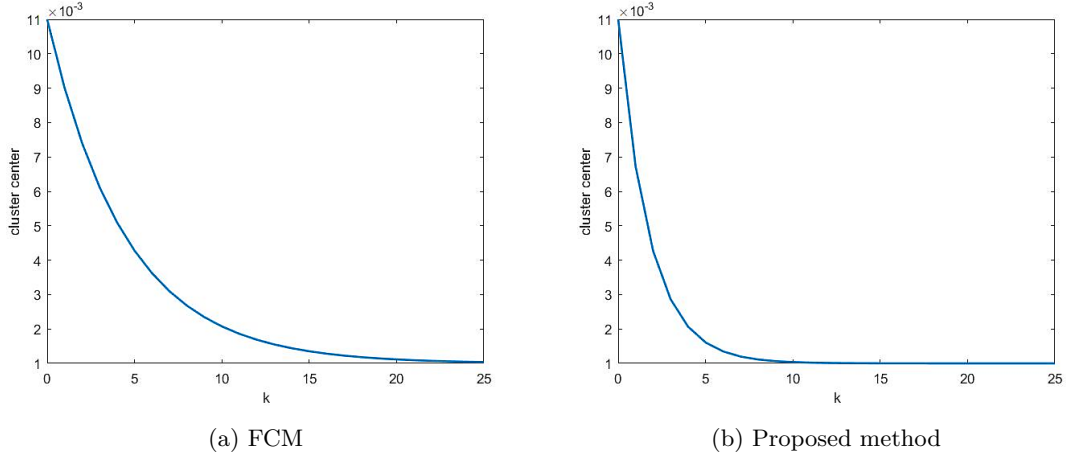


Figure 4: Comparison of empirical convergence rates of the proposed clustering method and FCM. Both follow the geometric decay predicted by Eq. (18); the proposed method attains a smaller effective contraction rate S , yielding faster convergence.

3.9 Rating prediction based on cluster membership and recommendation

Following the clustering of users based on their ratings and feature vectors, in the test phase, the next step is to predict each user's rating for a given item. If the target user is already part of the system, their cluster membership have been previously determined. For new users, these degrees are computed using the method outlined in the previous section. Once the user's cluster are determined, the predicted rating for a specific item is obtained as in Eq. 19 by averaging the ratings given to that item by the user's neighbors within the relevant clusters.

$$\frac{1}{C \times M} \sum_{k=1}^C \sum_{j=1}^M r_{kj} \quad (19)$$

Finally, after predicting the scores for each user-item pair, items with the highest predicted scores are selected for recommendation. In most recommender systems, it is common to recommend the top 5 or 10 items. In the proposed recommender system, the top 5 items with the highest predicted ratings are recommended to each user.

3.9.1 Convergence of the proposed method

According to the theorem of **hathaway1986local**, for a constant $m > 1$, the corresponding FCM algorithm is locally converge to every minimum point (U^*, μ^*) of J_m provided that the Hessian matrix of J_m is positive definite in all feasible directions. Hence, according to objective function defined in ??, Hessian matrix defined as :

$$H_J = \begin{bmatrix} \frac{\partial^2 J}{\partial u_{11}^2} & \frac{\partial^2 J}{\partial u_{11} \partial u_{12}} & \cdots & \frac{\partial^2 J}{\partial u_{11} \partial u_{NC}} & \frac{\partial^2 J}{\partial u_{11} \partial \theta_1} & \cdots & \frac{\partial^2 J}{\partial u_{11} \partial \theta_C} \\ \frac{\partial^2 J}{\partial u_{12} \partial u_{11}} & \frac{\partial^2 J}{\partial u_{12}^2} & \cdots & \frac{\partial^2 J}{\partial u_{12} \partial u_{NC}} & \frac{\partial^2 J}{\partial u_{12} \partial \theta_1} & \cdots & \frac{\partial^2 J}{\partial u_{12} \partial \theta_C} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 J}{\partial u_{NC} \partial u_{11}} & \frac{\partial^2 J}{\partial u_{NC} \partial u_{12}} & \cdots & \frac{\partial^2 J}{\partial u_{NC}^2} & \frac{\partial^2 J}{\partial u_{NC} \partial \theta_1} & \cdots & \frac{\partial^2 J}{\partial u_{NC} \partial \theta_C} \\ \frac{\partial^2 J}{\partial \theta_1 \partial u_{11}} & \frac{\partial^2 J}{\partial \theta_1 \partial u_{12}} & \cdots & \frac{\partial^2 J}{\partial \theta_1 \partial u_{NC}} & \frac{\partial^2 J}{\partial \theta_1^2} & \cdots & \frac{\partial^2 J}{\partial \theta_1 \partial \theta_C} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 J}{\partial \theta_C \partial u_{11}} & \frac{\partial^2 J}{\partial \theta_C \partial u_{12}} & \cdots & \frac{\partial^2 J}{\partial \theta_C \partial u_{NC}} & \frac{\partial^2 J}{\partial \theta_C \partial \theta_1} & \cdots & \frac{\partial^2 J}{\partial \theta_C^2} \end{bmatrix} \quad (20)$$

Then, to demonstrate that the Hessian matrix H_J is positive definite, we need to show that for every non-zero vector x , the value of $x^T H_J x$ is positive **bhatia2009positive**. The $x^T H_J x$ matrix includes four type of elements.

The expressions 21 and 22 are equal to zero:

$$x_k \frac{\partial^2 J}{\partial u_{ij} \partial u_{lh}} x_g = 0 \quad \forall i \neq l, j \neq h \quad (21)$$

$$x_k \frac{\partial^2 J}{\partial u_{ij} \partial \theta_l} x_g = x_k \frac{\partial^2 J}{\partial \theta_l \partial u_{ij}} x_g = 0 \quad \forall j \neq l \quad (22)$$

According to equations 23 and 23, the values of expressions 21 and 21 are equal to zero, respectively.

$$x_k \frac{\partial^2 J}{\partial u_{ij} \partial u_{lh}} x_g = x_k \left[\frac{\partial(m u_{ij}^{m-1} f(\theta_j | z_i))}{\partial u_{lh}} \right] x_g = 0 \quad (23)$$

$$x_k \frac{\partial^2 J}{\partial u_{ij} \partial \theta_l} x_g = x_k \left[\frac{\partial(m u_{ij}^{m-1} f(\theta_j | z_i))}{\partial \theta_l} \right] x_g = 0 \quad (24)$$

Also, the values of expressions like 25 and 26 are positive:

$$x_k \frac{\partial^2 J}{\partial u_{ij}^2} x_k = x_k^2 m(m-1) u_{ij}^{m-2} f(\theta_j | z_i) \geq 0 \quad (25)$$

$$\begin{aligned} x_{NC+j} \frac{\partial^2 J}{\partial \theta_j^2} x_{NC+j} &= \sum_{i=1}^N \frac{u_{ij}^m}{(2\pi)^{1/2} |\Sigma_j|^{1/2} |\Sigma_{\theta_j}|^{1/2}} \\ &\times \left[\Sigma_j^{-1}(z_i - \mu_j) + \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) \right]^2 \\ &\times \exp \left(-\frac{1}{2} \left((z_i - \mu_j)^T \Sigma_j^{-1}(z_i - \mu_j) + (\mu_j - \mu_{\theta_j})^T \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) \right) \right) \geq 0 \end{aligned} \quad (26)$$

In Equation 25, since $m > 1$, it follows that $m-1 > 0$. Also, $u_{ij} \geq 0$ because u_{ij} represents a membership degree, which can only be 0 or 1. Furthermore, $f(\theta_j | x_i) \geq 0$ as f is a posterior distribution function and therefore represents a probability.

In Eq. 26, in the first term, since the covariance matrix is symmetric and positive semi-definite, then $|\Sigma_j|^{1/2}$ and $|\Sigma_{\theta_j}|^{1/2}$ are positive, because the determinant of a matrix is the product of its eigenvalues, and in a positive semi-definite matrix, the eigenvalues are non-negative. The second and third terms are positive due to the squared terms and the exponential function.

Additionally, the sum of similar terms such as in Eq. 24 also results in zero:

$$\begin{aligned} &x_{NC+k} \frac{\partial^2 J}{\partial u_{ij} \partial \theta_j} x_g + x_g \frac{\partial^2 J}{\partial \theta_j \partial u_{ij}} x_{NC+k} \\ &= x_{NC+k} x_g \frac{m u_{ij}^{m-1}}{2\pi |\Sigma_{\theta_j}|^{1/2}} f_{ij} \left[|\Sigma_j|^{-5/2} \Sigma_j^{-T} \Sigma_j^{-1}(z_i - \mu_j) + |\Sigma_j|^{-5/2} \Sigma_j^{-T} \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) \right. \\ &\quad - \frac{1}{|\Sigma_j|^{1/2}} \Sigma_j^{-1}(z_i - \mu_j)^T \Sigma_j^{-1} + \frac{1}{|\Sigma_j|^{1/2}} \left(\Sigma_j^{-1}(z_i - \mu_j) + \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) \right)^2 \\ &\quad - |\Sigma_j|^{-5/2} \Sigma_j^{-T} \Sigma_j^{-1}(z_i - \mu_j) - \frac{1}{|\Sigma_j|^{1/2}} \left(\Sigma_j^{-1}(z_i - \mu_j) + \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) \right)^2 \\ &\quad \left. - |\Sigma_j|^{-5/2} \Sigma_j^{-T} \Sigma_{\theta_j}^{-1}(\mu_i - \mu_{\theta_j}) + \frac{1}{|\Sigma_j|^{1/2}} \Sigma_j^{-1}(z_i - \mu_j)^T \Sigma_j^{-1} \right] = 0 \end{aligned} \quad (27)$$

Finally, by substituting the expressions, the $x^T H_J x$ matrix is obtained as:

$$\begin{aligned}
x H_J^T x = & \left[x_1 \frac{\partial^2 J}{\partial u_{11}^2} x_1 + 0 + \dots + 0 + x_{NC+1} \frac{\partial^2 J}{\partial \theta_1 \partial u_{11}} x_1 + 0 + \dots + 0 \right] \\
& + \left[0 + x_2 \frac{\partial^2 J}{\partial u_{12}^2} x_2 + 0 + \dots + 0 + x_{NC+2} \frac{\partial^2 J}{\partial \theta_2 \partial u_{12}} x_2 + 0 + \dots + 0 \right] \\
& + \dots \\
& + \left[0 + \dots + 0 + x_{NC} \frac{\partial^2 J}{\partial u_{NC}^2} x_{NC} + 0 + \dots + 0 + x_{NC+C} \frac{\partial^2 J}{\partial \theta_C \partial u_{NC}} x_{NC} \right] \\
& + \left[x_1 \frac{\partial^2 J}{\partial u_{11} \partial \theta_1} x_{NC+1} + 0 + \dots + 0 + x_{NC+1} \frac{\partial^2 J}{\partial \theta_1^2} x_{NC+1} + 0 + \dots + 0 \right] \\
& + \dots \\
& + \left[0 + \dots + 0 + x_{NC} \frac{\partial^2 J}{\partial u_{NC} \partial \theta_C} x_{NC+C} + 0 + \dots + 0 + x_{NC+C} \frac{\partial^2 J}{\partial \theta_C^2} x_{NC+C} \right] \\
& \geq 0
\end{aligned} \tag{28}$$

The positivity of Eq. 28 confirms that the Hessian matrix of function H_J is positive definite in all directions. The theorem of **hathaway1986local** implies that if the clustering algorithm starts with points sufficiently close to the minimum of the objective function, it will converge after iterations. Since the proposed method utilizes prior knowledge, an initial clustering is performed based on the available data. This step ensures that the algorithm begins near the minimum point of the objective function, thereby it guarantees convergence.

3.9.2 Convergence rate of proposed method

To determine the convergence rate of the proposed clustering algorithm, the variations of cluster centroids across consecutive iterations were analyzed. First, we show that equation (29), which defines the cluster centroids, is a recursive relation based on the cluster centroids in the previous iteration:

$$\mu_{n+1} = \left(\frac{\Sigma_j^{-1}}{\Sigma_j^{-1} + \Sigma_{\theta_j}^{-1}} \right) \left(\frac{\sum_{i=1}^n u_{ij}^m f_{ij} x_i}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}} + \frac{u_{(n+1)j}^m f_{(n+1)j} x_{n+1}}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}} \right) + \left(\frac{\Sigma_{\theta_j}^{-1}}{\Sigma_j^{-1} + \Sigma_{\theta_j}^{-1}} \right) \frac{\sum_{i=1}^n u_{ij}^m f_{ij} \mu_{\theta_j}}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}} \tag{29}$$

this leads to the following simplified form:

$$\mu_{n+1} = \mu_n \cdot S + x_{n+1} \cdot (1 - S) + M \tag{30}$$

where

$$S = \frac{\sum_{i=1}^n u_{ij}^m f_{ij}}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}}, \quad 1 - S = \frac{u_{(n+1)j}^m f_{(n+1)j}}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}}, \quad M = \left(\frac{\Sigma_{\theta_j}^{-1}}{\Sigma_j^{-1} + \Sigma_{\theta_j}^{-1}} \right) \frac{\sum_{i=1}^n u_{ij}^m f_{ij} \mu_{\theta_j}}{\sum_{i=1}^{n+1} u_{ij}^m f_{ij}} \tag{31}$$

The differential of $\mu(k)$ is considered, to investigate the change in cluster centroids during each iteration. Based on the recursive equation for the cluster centroid, the variation resembles the capacitor discharge equation. According to Kirchhoff's law, the voltage drop across a resistor-capacitor circuit is modeled by a first-order differential equation:

$$\frac{Q}{C} + R \frac{dQ}{dt} = 0 \Rightarrow \frac{dQ}{dt} = -\frac{Q}{\tau}, \quad \text{where } \tau = RC \tag{32}$$

integrating both sides yields:

$$\int_{Q=Q_0}^{Q=Q(t)} \frac{dQ}{Q} = - \int_{t=0}^t \frac{dt}{\tau} \tag{33}$$

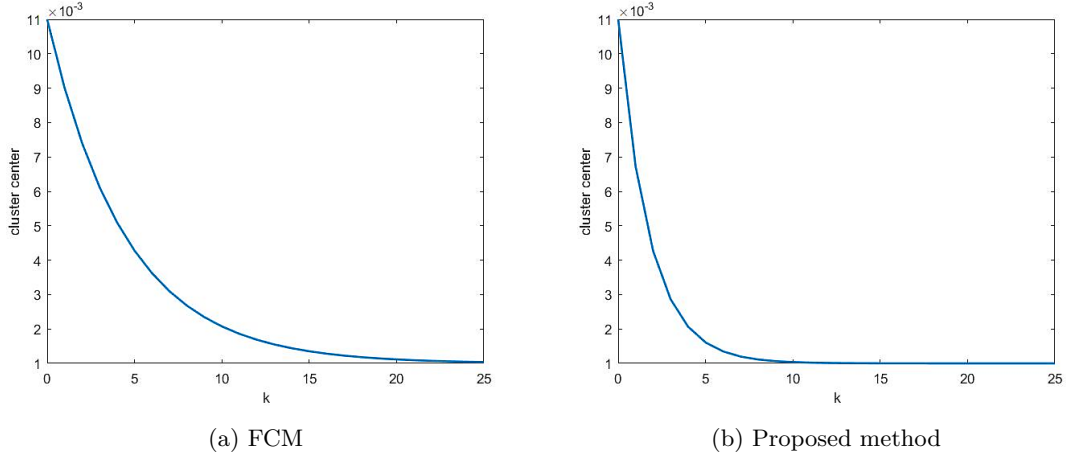


Figure 5: Comparison of convergence rates of the proposed clustering method and FCM method

$$\ln \left[\frac{Q(t)}{Q_0} \right] = -\frac{t}{\tau} \quad (34)$$

Similar to the capacitor discharge equation, the differential of $\mu(k)$ in the continuous case becomes:

$$\frac{d\mu}{dt} = -\frac{\mu}{\tau} \quad (35)$$

integrating leads to:

$$\mu(k) = \mu(0)e^{-t/\tau} \quad (36)$$

in the discrete case:

$$\frac{\mu(k) - \mu(0)}{k} = -\frac{\mu(k)}{\tau} \Rightarrow \tau_\mu = \frac{-k\mu(k)}{\mu(k) - \mu(0)} \quad (37)$$

then, $\mu(k)$ is obtained recursively as:

$$\mu(k) = \mu(0) + \sum_{i=1}^k S^{k-i}(1-S)x(i) + M, \quad M \triangleq kM \quad (38)$$

substituting $\mu(k)$ into Eq. 38, have:

$$\tau_\mu = \frac{-k\mu(0)}{\sum_{i=1}^k S^{k-i}(1-S)x(i) + M} + k \quad (39)$$

Finally, substituting this into, the exponential model for the variation of cluster centroids in the proposed method is obtained as:

$$\mu(k) = \mu(0)e^{-t/\frac{-k\mu(0)}{\sum_{i=1}^k S^{k-i}(1-S)x(i) + M} + k} \quad (40)$$

In Figure 5, the exponential models of the proposed method and the FCM method are compared. The figure indicates that the cluster centroids in the proposed method change more rapidly, resulting in a faster convergence compared to FCM.

4 Experiments

In this section, we investigate the accuracy and performance of the proposed method on various datasets. A comparative analysis with other methods as well as a discussion in parameter setting is presented.

4.1 Data sets

The proposed recommender system was evaluated on the widely used MovieLens-100k, and MovieLens-1M and Yelp datasets, which are standard benchmarks in recommender system methods. The MovieLens dataset includes user ratings and movie features collected from the MovieLens website, with movies categorized by genre. Preprocessing converted genres into binary indicators, where 1 denotes genre membership and 0 otherwise. The study utilizes MovieLens 100K, which includes 100,000 ratings from 1,000 users across 1,700 movies, and MovieLens 1M, containing 1 million ratings from 6,000 users for 4,000 movies, with additional user attributes such as age, gender, and occupation. These attributes are used for clustering in recommendation models.

The Yelp dataset features information on businesses across 11 cities in 4 countries, focusing on the restaurant category. It includes 4 million reviews from 1 million users for 58,000 restaurants, with detailed restaurant features (e.g., parking, child-friendliness, ambiance) and user attributes (e.g., age, gender, and average rating). Restaurant features were converted into numerical values (0 for absence, 1 for presence, and -1 for unspecified).

4.2 Evaluation criteria

Recommender systems are typically evaluated using two categories of criteria, prediction accuracy and decision support. Prediction accuracy metrics, such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), measure the numerical accuracy of predicted ratings.

$$\text{MAE} = \frac{1}{|S|} \sum_{i=1}^{|S|} |pred_i - r_i| \quad (41)$$

$$\text{RMSE} = \sqrt{\frac{1}{|S|} \sum_{i=1}^{|S|} (pred_i - r_i)^2} \quad (42)$$

where, $pred_i$ and r_i are the predicted and real scores for item i respectively, and $|S|$ is number of test data.

In contrast, decision support metrics, including precision, recall, and F1 score, evaluate the system's effectiveness in providing relevant recommendations. Predicted scores for the items determined whether an item should be considered relevant and recommended to the target user or not. With a score range of 1 to 5, items rating ≤ 3 are classified as irrelevant, while those rating ≥ 4 are considered relevant. Therefore, precision, defined as the ratio of relevant suggested items to the total suggested items, ensures that recommendations are relevant and minimize unnecessary suggestions. Recall is the ratio of suggested relevant items to the total number of relevant items. F1 is a combination of precision and recall. These are defined as follows

$$\text{Precision} = \frac{\#tp}{\#tp + \#fp}$$

$$\text{Recall} = \frac{\#tp}{\#tp + \#fn}$$

$$F1 = \frac{2 \times P \times R}{P + R}$$

where, tp denotes the items suggested to the user by the recommendation system based on the predicted score, that are also relevant according to the actual score. Similarly, fp, fn and tn represent the suggested irrelevant items, not suggested relevant items and not suggested irrelevant items, respectively.

4.3 Evaluation of the Proposed Recommender System

To assess the accuracy of the proposed recommender system and compare the effectiveness of the proposed approach, its performance was benchmarked against existing clustering-based recommendation methods, including **liji2018improved**, **kant2017leaderrank**, **mohammadpour2019efficient**, **wasid2018clustering**.

Table 1 presents the comparison results for different methods using the evaluation criteria on the MovieLens-100K, MovieLens-1M, and Yelp datasets.

Table 1: Comparison of the Proposed Method with others

Data Sets	Method	MAE	RMSE	Precision	Recall	F1
ML_100K	Chen liji2018improved	0.74	0.95	0.88	0.50	0.642
	Kant kant2017leaderrank	0.75	0.95	0.56	0.87	0.681
	Enayatifar mohammadpour2019efficient	0.78	0.96	-	-	-
	Wasid wasid2018clustering	0.91	1.12	0.54	0.75	0.627
	Proposed Method	0.71	0.91	0.56	0.89	0.687
ML_1M	Chen	0.72	0.94	85.3	0.49	0.622
	Kant	0.72	0.925	0.53	0.86	0.653
	Enayatifar	0.85	1.35	-	-	-
	Wasid	0.95	1.24	0.56	0.71	0.626
	Proposed Method	0.705	0.925	0.62	0.85	0.71
Yelp	Kant	0.965	1.25	0.57	0.78	0.65
	Proposed Method	0.943	1.21	0.61	0.81	0.69

In all the mentioned methods, a clustering model has been used to find users or similar items and finally predict the score for the users. Also, in all these studies, the number of data has been assumed to be constant. According to the comparison, it is observed that although the proposed method considers the data as online and streaming, it has better performance than other methods.

Furthermore, to evaluate the feasibility of the proposed method for real-time applications, we analyzed the computation time required for rating predictions. Table 2 presents the comparison between the proposed method and the approach **kant2017leaderrank** in terms of maximum computational time required per prediction in three datasets. It is evident that the proposed method performs efficiently compared to existing approaches. Its speed makes it suitable for deployment in real-world systems with streaming data.

Table 2: Computational Time Comparison (in milliseconds)

Method	Yelp	MovieLens-100K	MovieLens-1M
Kant et al. [43]	48 ms	18 ms	25 ms
Proposed Method	50 ms	22 ms	28 ms

4.4 comparison with standard FCM

For comparing the final performance and convergence speed of the proposed clustering method versus the standard FCM, the FCM objective function's value is computed separately for both methods per iteration. This allows for a proper comparison, even though each method updates parameters differently and has its own unique objective function. The convergence speed comparison for every dataset can be seen in Figure 6.

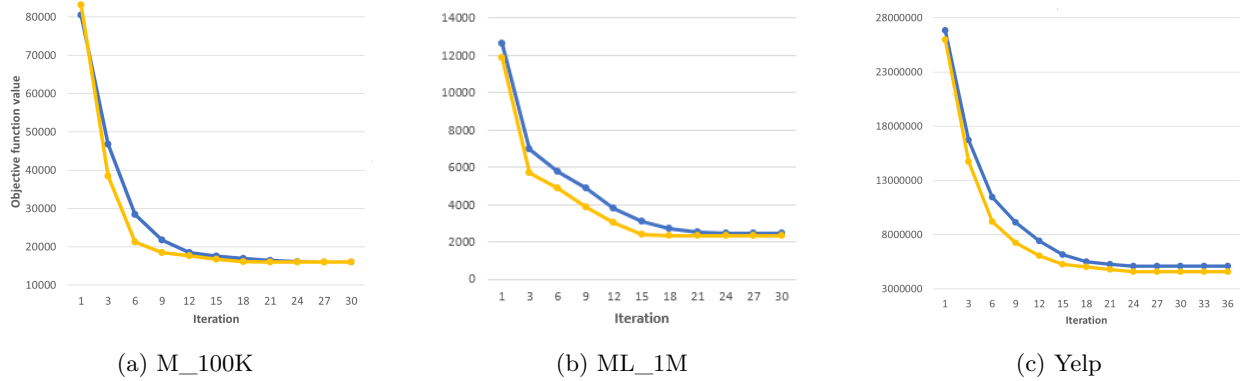


Figure 6: Comparison of the convergence rate and speed of the proposed method and the FCM method on the different datasets. Blue and yellow line denotes the FCM and proposed method respectively

Table 3 compares both methods based on the minimum FCM objective function value attained and the minimum iterations required. The comparison demonstrates that the proposed method outperforms FCM by converging faster and lowering the objective function value more substantially.

Table 3: Comparison of the proposed method with FCM

Data set	FCM		Proposed method	
	value	iterations	value	iterations
ML-100K	2709	18	2402	15
ML-1M	16099	22	16093	19
Yelp	5078242	26	4587557	23

4.5 Parameter Discussion

To assess the effect of different parameter settings on performance, we conducted experiments to analyze the impact of number of cluster and the percentage of data used in initial clustering. Initially, 80% of the data is randomly selected as training data, while the remaining 20% is used for testing. Then, a portion of the training data is allocated for initial clustering and parameter setting. Consequently, both the number of user clusters and the proportion of data used for this initial phase influence the quality of the clustering results. Table 4 presents the prediction accuracy of the proposed recommender system across different numbers of user clusters and considering two data usage scenarios: 30% and 40% of the training data.

The results indicate that using 40% of the data for initial clustering leads to more effective user clustering, which enhances the overall prediction accuracy.

In addition, based on these comparisons, the optimal number of clusters for users in each data set is determined. According to the results obtained, the optimal number of clusters for the MovieLens_100k data set is 80 clusters, for the MovieLens_1M data set is 100 clusters, and for the yelp data set is 110 clusters.

Also, according to the experiments conducted, the optimal number of clusters for items in the MovieLens dataset, due to the limited number of genres, was 20 clusters and in the Yelp dataset, 80 clusters.

These findings confirm that the proposed Bayesian clustering model outperforms traditional clustering approaches while maintaining low computational overhead, making it well-suited for real-time recommender systems.

Table 4: Accuracy of the proposed method across different parameter settings

Datasets	initial	60		80		100		110	
	data	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
ML_100K	30%	0.73	0.925	0.72	0.92	0.76	0.955	-	-
	40%	0.73	0.917	0.71	0.91	0.74	0.96	-	-
ML_1M	30%	0.78	1.0	0.763	0.979	0.72	0.95	0.725	0.935
	40%	0.77	0.98	0.752	0.976	0.705	0.925	0.718	0.93
Yelp	30%	0.993	1.45	0.975	1.43	0.965	1.31	0.955	1.32
	40%	0.99	1.47	0.965	1.40	0.945	1.25	0.943	1.21

5 Conclusion

In this study, a method based on the Bayesian clustering model is used. Initially, to reduce the feature for user clustering, the items are clustered using FCM. Then, As considering the users are not constant and added to the system dynamically, a Bayesian clustering method is used for user clustering. In this approach, learning is conducted in a distributed manner, where cluster centroids interact and influence each another based on their relative distances. This minimizes the impact of outliers on the cluster centroids. Finally, after clustering, the scores of neighboring users are used to predict a user’s score on an item that they have not seen yet, and the items with the highest scores are suggested to each user. The evaluation results on streaming data events demonstrate that the proposed method outperforms existing approaches in enhancing recommender system performance, achieving more precise user score predictions. In proposed method, items are considered constant, which is a limitation in real-world application. Future research could address this by considering dynamic item. Furthermore, integrating a score prediction methods that utilize both user-user and item-item similarities could enhance recommendation accuracy.